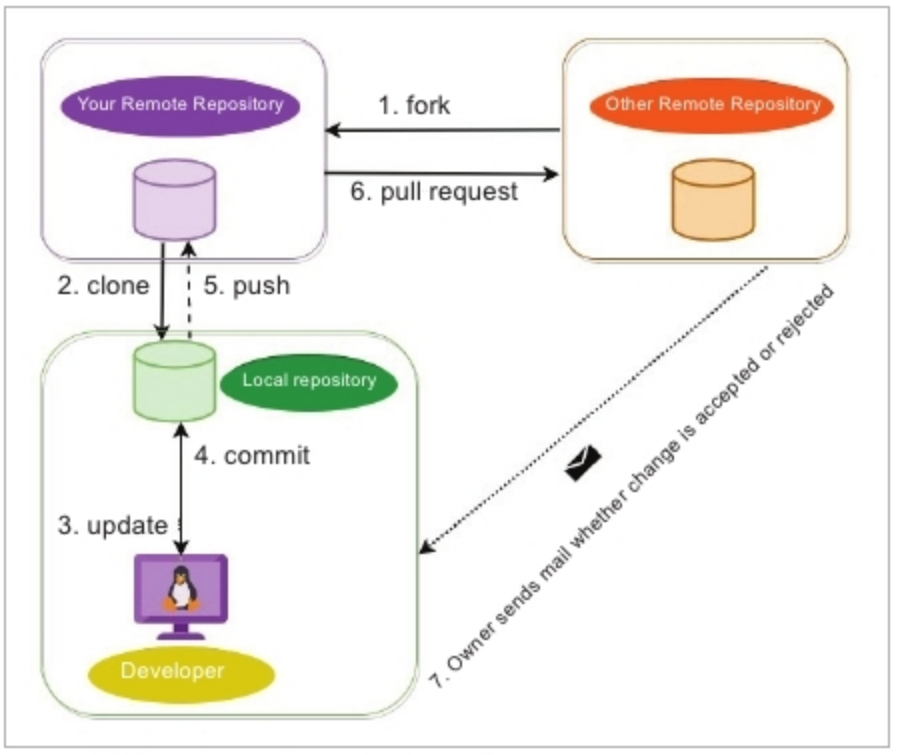

Figure 14: Block diagram demonstrating major commands

default would be the URL from which it was cloned with the shorthand as 'origin'.

To add a new remote repository, the following command is used:

```
$ git remote add <shorthand> <url-to-remote-repo>
```

We have created a repository on GitHub to demonstrate working with remote repositories in Git, as shown in Figure 10.

Pushing the changes to the remote repository: The *git push* command is used to update the remote repository with the changes made in the local repository. It acts as a syncing function that uploads the local changes made in the codebase to the remote repository.

The basic command used is: *git push <remote> <branch>*. Here *<remote>* is the destination and *<branch>* is the source of the *push* command. An example of this could be: *git push origin master.* Here the *master* branch from the local repository is pushed to the origin (which is shorthand for the remote repository).

One thing to note is that *git push* only works when there is a fast-forwarding merge in the destination repository. A merge or any reference change in Git is called *fast-forward* if it is possible to traverse back to the old reference from the new reference. So, if the old reference is an ancestor of the new reference then it's a *fast-forward*, which is explained in Figure 11.

Fetching the changes from the remote repository: The *git fetch* command downloads files, commits, refs, etc, from a remote repository to a local repository. The *fetch* command only downloads and doesn't merge into the local repository. It has absolutely no effect on the local development. The head and the refs are not changed and no new *merge commit* is created. One can check out the *fetch* content and see the changes in order to decide whether to merge it or not. The basic structure of the command is: *git fetch <remote>* where *<remote>* is the source remote repository.

Pulling the changes from the remote repository: The *git pull* command is used to pull in all the changes made in the remote repository to the local repository. The basic structure of the command is *git pull <remote> <branch-name>* which is shown in Figure 12.

The *git pull* command undergoes two internal operations: *git fetch* and *git merge*. *git fetch*, as described earlier, is used to download the contents of a branch from the specified remote repository. Then the *git merge* command is run, which merges the remote refs and heads into a new local *merge commit*. Instead of *merge*, *rebase* can also be done after *fetch*. For this, the *git pull --rebase <remote> <branch-name>* command is used.

Collaborating with open source projects

Every project/organisation has its own process with respect to collaboration. Forking a repository is one of the ways to collaborate with someone on a project or an open source project. For a demo, the OVS project at *https://github.com/openvswitch/openvswitch.github.io*, which is forked from GitHub, is shown in Figure 13.

Once the project is forked, it will be present in our GitHub account. The next steps are cloning it in a local machine, working on it like any other project and pushing the changes to the cloned repository. If we wish to contribute those changes back to the original repository, a *pull* request can be created with the proposed changes. The maintainer of the project will decide whether to accept or reject the proposed changes, and will send a notification regarding the same, as shown in Figure 14.

To summarise, Git provides developers a robust version control system which helps them in doing non-linear and parallel development. Because of its lightweight branches and consistent tracking, it provides scalability as well as immunity from data loss. These facts make Git an absolute favourite among developers. END

Reference

The entire Pro Git book, written by Scott Chacon and Ben Straub and published by Apress *(https://git-scm.com/book/en/v2)*

By: Shubham Kumar, Ekta Nandwani and Prof. B. Thangaraju

The authors are associated with the Open Source Technology Lab at the International Institute of Information Technology, Bengaluru.